

Backend Components

IE University - BCSAI - SDD - 2025

Introduction

Modern backend systems are built from **numerous interconnected components**, each serving specific purposes.

Understanding these components and how they work together is crucial for designing:

- **Scalable** systems
- **Reliable** systems
- **Maintainable** systems

Overview of Component Categories

1. **Traffic & Entry** - DNS, CDN, API Gateway
2. **Networking** - Load Balancers, Firewalls, Protocols
3. **Compute** - VMs, Containers, Serverless
4. **Data: Databases** - Relational, NoSQL, Time-Series
5. **Data: Storage** - Object, File, Block, Archive
6. **Messaging & Async** - Message Brokers, Event-Driven
7. **Caching & Edge** - In-Memory, HTTP, Edge Computing
8. **Jobs & Scheduling** - Cron, Task Queues, Workflows

Traffic & Entry

Traffic & Entry

The **first line of contact** between users and your backend systems.

These components manage:

- DNS resolution
- Content delivery
- API routing
- Load balancing
- Security at the edge

They form the **critical gateway** that determines how users access your applications.

DNS (Domain Name System)

Translates **human-readable domain names** (like `example.com`) into **IP addresses**.

Key Features:

- Distributed, hierarchical database
- Caching for improved performance
- Multiple query types (A, AAAA, CNAME, MX, TXT, NS, SRV)
- Can be used for load balancing (geographic, health checks)

Common Solutions:

- Open-Source: BIND, CoreDNS
- Cloud: AWS Route 53, Azure DNS, GCP Cloud DNS

CDN / Edge

Content Delivery Networks distribute content globally to reduce latency.

How it works:

- Replicate content across geographically distributed servers
- Reduce distance data must travel
- Offload traffic from origin servers

Advanced Features:

- DDoS protection & Web Application Firewalls
- Edge computing capabilities
- SSL/TLS termination
- Cache control & invalidation

CDN Solutions

Open-Source / Self-Hosted:

- Apache Traffic Server - high-performance caching proxy
- Varnish Cache - HTTP accelerator
- Nginx - edge cache with proper configuration

Third-Party Providers:

- Cloudflare - security-focused
- Fastly - real-time, instant purging
- Akamai - enterprise, media streaming

Cloud Providers:

- AWS CloudFront, Azure Front Door, GCP Cloud CDN

API Gateway / Reverse Proxy

Sit between **clients** and **backend services**.

API Gateway provides:

- Single entry point for client applications
- Authentication, rate limiting, request transformation
- Service discovery, protocol translation
- API versioning

Reverse Proxy provides:

- SSL termination
- Caching
- Request/response modification

Gateway Solutions

Open-Source / Self-Hosted:

- **Nginx** - high-performance web server and reverse proxy
- **Kong** - API Gateway with extensive plugin ecosystem
- **Traefik** - cloud-native with automatic service discovery

Cloud Provider Solutions:

- AWS API Gateway/ALB/NLB
- Azure API Management/Application Gateway
- GCP API Gateway/Cloud Load Balancing

Networking

Networking

Essential for ensuring **data flows efficiently and securely** between different parts of a backend system.

Key Components:

- **Load Balancers** - distribute traffic
- **Firewalls** - security enforcement
- **Network Protocols** - communication standards

Load Balancers

Distribute incoming network traffic across **multiple backend servers**.

Layer 4 (L4):

- TCP/UDP connections
- Routing based on network information
- Maximum performance

Layer 7 (L7):

- Application layer
- Intelligent routing (HTTP headers, cookies, URLs)
- Content-based routing

Load Balancer Features

Core Capabilities:

- Health checking (remove failed servers)
- SSL termination (offload encryption)
- Session persistence (maintain user state)
- Algorithms: round-robin, least connections, IP hash

Essential for:

- Horizontal scalability
- High availability
- Zero-downtime deployments

Load Balancer Solutions

Open-Source / Self-Hosted:

- **HAProxy** - industry-standard TCP/HTTP load balancer
- **Nginx** - web server with load balancing
- **Envoy** - modern L7 proxy with observability

Cloud Provider Solutions:

- AWS Elastic Load Balancing
- Azure Load Balancer/Traffic Manager
- GCP Cloud Load Balancing

Firewalls

Security enforcement points that inspect and filter network traffic.

Network Firewalls (L3/L4):

- Protect entire network segments
- Stateful inspection (track connection states)
- IP addresses, ports, protocols

Web Application Firewalls (WAF) (L7):

- Protect specific applications
- SQL injection, XSS, OWASP Top 10
- Deep packet inspection

Firewall Solutions

Open-Source / Self-Hosted:

- **iptables/nftables** - Linux kernel-level packet filtering
- **pfSense** - full-featured firewall with web interface
- **ModSecurity** - WAF engine for Apache/Nginx

Cloud Provider Solutions:

- AWS WAF/Network Firewall/Security Groups
- Azure Firewall/WAF/NSGs
- GCP Cloud Armor/VPC Firewall/Cloud IDS

Network Protocols

Standardized rule sets that govern how data is communicated.

Key Protocols:

- **TCP** - reliable, ordered delivery
- **UDP** - low-latency, connectionless
- **HTTP/HTTPS** - request-response web communication
- **WebSocket** - persistent bidirectional connections
- **gRPC** - efficient RPC over HTTP/2
- **QUIC** - next-generation transport protocol

Compute

Compute

Provide the **processing power** required to run applications and services.

Three main paradigms:

1. **Virtual Machines (VMs)** - full OS control
2. **Containers** - lightweight, portable
3. **Serverless / FaaS** - no infrastructure management

Each has different trade-offs in **control**, **efficiency**, and **operational complexity**.

Virtual Machines (VMs)

Software-based emulations of physical computers.

Characteristics:

- Run complete operating systems
- Strong isolation
- Full OS-level control
- More overhead than containers (full OS kernel)

Best for:

- Legacy applications
- Specific OS versions or kernel modules
- Complete environment control

VM Solutions

Open-Source / Self-Hosted:

- **KVM** - Linux kernel virtualization
- **Proxmox VE** - complete virtualization management platform
- **VMware ESXi** - enterprise hypervisor

Cloud Provider Solutions:

- AWS EC2
- Azure Virtual Machines
- GCP Compute Engine

Features: varied instance types, auto-scaling, high availability

Containers

Lightweight, standalone packages that bundle application code with all dependencies.

Key Differences from VMs:

- Share host OS kernel
- Significantly smaller (MB vs GB)
- Faster to start (seconds vs minutes)
- Process-level isolation (namespaces, cgroups)

Benefits:

- Consistent development-to-production environments
- Efficient resource utilization
- Rapid scaling

Container Solutions

Open-Source / Self-Hosted:

- **Docker** - industry-standard container runtime
- **Kubernetes** - production-grade orchestration
- **Docker Swarm** - simpler orchestration

Cloud Provider Solutions:

- AWS ECS/EKS/Fargate
- Azure AKS/Container Instances/Container Apps
- GCP GKE/Cloud Run

Kubernetes Orchestration

Kubernetes manages:

- **Deployment** - rolling updates, rollbacks
- **Scaling** - horizontal pod autoscaling
- **Networking** - service discovery, load balancing
- **Lifecycle** - health checks, self-healing
- **Storage** - persistent volumes

Enables **microservices architectures** at scale.

Serverless / Function as a Service (FaaS)

Abstract infrastructure management entirely.

Key Characteristics:

- Deploy code as functions responding to events
- No provisioning or managing servers
- Automatic scaling (zero to thousands)
- Pay only for actual execution time

Benefits:

- Eliminate server maintenance
- Reduced costs for variable workloads
- Focus purely on business logic

Serverless Constraints

Limitations to consider:

- Execution time limits
- Cold start latency
- Statelessness requirements
- Potential vendor lock-in

Best for:

- Event-driven architectures
- APIs with unpredictable traffic
- Scheduled tasks
- Data processing pipelines

Serverless Solutions

Open-Source / Self-Hosted:

- **OpenFaaS** - Docker/Kubernetes-based
- **Knative** - Kubernetes-native
- **Apache OpenWhisk** - distributed serverless

Cloud Provider Solutions:

- AWS Lambda
- Azure Functions
- GCP Cloud Functions

Compute: When to Use

- **Use VMs** for full control, legacy apps, predictable workloads
- **Use containers** for microservices, portability, consistent deployment
- **Use serverless** for event-driven workloads, variable traffic, minimal ops

Choose cloud solutions for:

- Managed infrastructure
- Automatic scaling
- Reduced maintenance

Data: Databases

Data: Databases

Critical component of backend systems for storing, retrieving, and managing data.

Different types suited for different use cases:

1. **Relational** - structured data, complex queries
2. **NoSQL** - flexible schemas, horizontal scaling
3. **Time-Series** - time-stamped metrics and events

Relational Databases

Organize data into **tables with rows and columns**.

Key Features:

- Structured schemas (data types, relationships, constraints)
- SQL for declarative querying
- **ACID** guarantees (Atomicity, Consistency, Isolation, Durability)
- Complex queries with joins
- Referential integrity (foreign keys)

Best for: transactional workloads, financial systems, structured data

Relational Database Features

Advanced Capabilities:

- B-tree indexes for fast lookups
- Query optimizers for efficient execution
- Transaction logs for crash recovery
- Replication for high availability

Modern distributed SQL:

- CockroachDB, Cloud Spanner
- Horizontal scalability + strong consistency

Relational Solutions

Open-Source / Self-Hosted:

- **PostgreSQL** - advanced SQL, JSON support, extensibility
- **MySQL/MariaDB** - high-performance, mature ecosystem
- **SQLite** - embedded, zero-configuration

Cloud Provider Solutions:

- AWS RDS/Aurora
- Azure SQL Database/PostgreSQL/MySQL
- GCP Cloud SQL/Cloud Spanner

Features: automatic backups, scaling, high availability

NoSQL Databases

Sacrifice **rigid structure** and **ACID** for:

- Flexible schemas
- Horizontal scalability
- Performance optimization for specific use cases

Types:

- **Document** - hierarchical, semi-structured
- **Key-Value** - simple high-speed lookups
- **Column-Family** - analytical queries
- **Graph** - relationship-heavy data

Document Databases

Store data as **JSON-like documents** with nested structures.

Advantages:

- Schema flexibility
- Store related data together
- No migrations for evolving requirements
- Rich queries and aggregation pipelines

Open-Source: MongoDB, CouchDB

Cloud: AWS DocumentDB, Azure Cosmos DB, GCP Firestore

Best for: general-purpose document storage, content management

Key-Value Stores

Simplest data model: **store and retrieve values by unique keys.**

Characteristics:

- Extremely fast lookups (microseconds)
- Minimal overhead
- Most are in-memory
- Persistence options available

Open-Source: Redis, etcd

Cloud: AWS DynamoDB, Azure Cosmos DB, GCP Memorystore

Best for: caching, session storage, real-time applications

Column-Family Stores

Organize data in **columns rather than rows**.

Optimized for:

- Analytical workloads
- Wide tables with sparse data
- Time-series data
- High-write throughput

Open-Source: Apache Cassandra, Apache HBase

Cloud: AWS Keyspaces, Azure Cosmos DB, GCP Bigtable

Best for: write-heavy workloads requiring horizontal scaling

Graph Databases

Store data as **nodes and edges**.

Optimized for:

- Relationship-heavy data
- Complex traversal queries
- Social networks
- Recommendation engines
- Fraud detection

Open-Source: Neo4j, ArangoDB, JanusGraph

Cloud: AWS Neptune, Azure Cosmos DB (Gremlin API)

Best for: highly connected data and relationship queries

Time-Series Databases

Optimized for **time-stamped data**.

Features:

- Efficient compression
- Downsampling
- Aggregation of temporal data

Open-Source: InfluxDB, TimescaleDB, Prometheus

Cloud: AWS Timestream, Azure Time Series Insights, GCP Bigtable

Best for: metrics, monitoring, sensor data, IoT

Choosing the Right Database

Consider:

- **Data Structure** - structured, semi-structured, unstructured
- **Scalability** - vertical vs horizontal
- **Consistency vs Availability** - CAP theorem trade-offs
- **Query Patterns** - complex joins vs simple lookups
- **Transactions** - ACID compliance vs eventual consistency
- **Performance** - read vs write heavy, latency requirements

Database Selection Guide

- **Relational** → structured data, complex queries, strong consistency
- **Document** → flexible schemas, hierarchical data
- **Key-Value** → simple lookups, caching, session management
- **Column-Family** → analytical workloads, time-series
- **Graph** → highly connected data, relationship queries

Cloud managed databases → automatic backups, scaling, reduced ops

Data: Storage

Data: Storage

Different storage types for different **data types and access patterns**.

Must balance:

- **Performance** (throughput, latency, IOPS)
- **Durability** (data protection)
- **Availability** (uptime)
- **Scalability** (growing data volumes)
- **Cost**

Object Storage

Store data as **discrete objects** in a flat address space.

Each object contains:

- The data itself
- Metadata (rich, extensible)
- Unique identifier (key)

Characteristics:

- Flat namespace (no hierarchy)
- Retrieved directly by key
- Massive scalability
- HTTP APIs (S3-compatible)

Object Storage Features

Advanced Capabilities:

- Versioning (multiple versions of objects)
- Lifecycle policies (automatic tiering/deletion)
- Event notifications (trigger actions)
- Multipart uploads (large files)

Open-Source: MinIO, Ceph, OpenStack Swift

Cloud: AWS S3, Azure Blob Storage, GCP Cloud Storage

Best for: media files, backups, logs, data lakes

File Storage

Hierarchical directory structure with folders and files.

Features:

- POSIX-compliant file system semantics
- File locking, permissions, random access
- Multiple clients access same file system
- Network protocols (NFS, SMB/CIFS)

Open-Source: NFS, Samba/SMB, GlusterFS

Cloud: AWS EFS/FSx, Azure Files, GCP Filestore

Best for: content management, shared directories, concurrent access

Block Storage

Present **raw storage volumes** as virtual disks.

Characteristics:

- Low-level, unformatted storage
- Operates at block level (512 bytes to 64KB blocks)
- Lowest latency and highest IOPS
- Appears as local disks to OS
- Strict consistency guarantees

Open-Source: LVM, Ceph RBD, iSCSI

Cloud: AWS EBS, Azure Managed Disks, GCP Persistent Disk

Best for: databases, VMs, high-performance applications

Archive Storage

Optimized for **long-term retention** of infrequently accessed data.

Trade-offs:

- Lower storage costs
- Slower retrieval (minutes to hours)
- Same durability (11 nines)

Use Cases:

- Compliance and regulatory retention
- Legal holds
- Historical backups

Cloud: AWS S3 Glacier/Deep Archive, Azure Archive Storage, GCP Archive

Storage Selection Guide

- **Object Storage** → unstructured data, backups, media, data lakes
- **File Storage** → shared file access, content management
- **Block Storage** → databases, VMs, low-latency applications
- **Archive Storage** → long-term retention, compliance

Cloud storage benefits:

- Managed infrastructure
- Automatic scaling
- Durability and global distribution

Messaging & Async

Messaging & Async

Enable services to communicate **without requiring both parties to be actively processing.**

Message brokers act as intermediaries:

- Receive, store, route, deliver messages
- Decouple producers and consumers
- Time and space decoupling

Benefits:

- Continue operating if consumers unavailable
- Independent scaling of components
- Natural buffers for traffic spikes

Delivery Semantics

Three guarantees:

1. **At-Most-Once** - fire and forget (may lose messages)
2. **At-Least-Once** - may deliver duplicates
3. **Exactly-Once** - guaranteed single delivery

Trade-offs between **performance** and **reliability**.

Communication Patterns

Point-to-Point (Queues):

- One consumer per message
- Work distribution and load leveling

Publish-Subscribe (Topics):

- Multiple consumers per message
- Broadcasting events

Request-Reply:

- Synchronous-style over async messaging

Message Brokers

Implement **queuing systems** that store messages until consumers ready.

Features:

- Persistence, acknowledgments, redelivery
- Message prioritization, TTL
- Batching, compression
- Message filtering, transformation

Different brokers optimize for:

- Throughput vs latency
- Ordering guarantees
- Operational complexity

Message Broker Solutions

Open-Source / Self-Hosted:

- **RabbitMQ** - AMQP, complex routing, delivery guarantees
- **Apache Kafka** - high-throughput, streaming, event sourcing
- **ActiveMQ** - enterprise messaging, multiple protocols

Cloud Provider Solutions:

- AWS SQS
- Azure Service Bus
- GCP Pub/Sub

Event-Driven Architectures

Structure systems around **production, detection, and reaction to events**.

Key Concepts:

- **Event Producers** - generate events
- **Event Consumers** - listen and process events
- **Event Streams** - continuous flows processed real-time

Patterns:

- **Event Sourcing** - store state changes as events
- **CQRS** - separate read/write models
- **Saga Pattern** - manage distributed transactions

Event-Driven Benefits

Advantages:

- Loose coupling (producers don't know consumers)
- Independent evolution of services
- Asynchronous processes
- Built-in audit trails
- Real-time analytics
- Reactive systems

Invert traditional request-response - services react to events passively.

Messaging: When to Use

- **Messaging** for decoupling services, handling spikes, reliable communication
- **Event-Driven** for real-time processing, audit trails, reactive systems
- **Point-to-Point** for work distribution
- **Publish-Subscribe** for broadcasting events
- **Cloud services** for managed solutions with minimal overhead

Caching & Edge

Caching & Edge

Performance optimization by storing data in faster, more accessible locations.

Benefits:

- Reduce latency
- Reduce computational load
- Reduce costs
- Improve user experience

Key Principle:

- **Temporal locality** - recently accessed data likely accessed again
- **Spatial locality** - nearby data also likely accessed

Cache Considerations

Critical aspects:

- **Cache Invalidation** - ensure stale data doesn't persist
- **Eviction Policies** - what to remove when cache full (LRU, LFU, TTL)
- **Cache Coherency** - maintain consistency across distributed caches

Multi-tier caching:

Browser → CDN Edge → Application → Database

In-Memory Caching

Store data in **RAM** for microsecond access times.

Performance:

- 100-1000x faster than database queries
- Milliseconds → microseconds

Use Cases:

- Reduce database load
- Store session state
- Rate limiting
- Leaderboards and counters

In-Memory Cache Solutions

Open-Source / Self-Hosted:

- **Redis** - data structures, persistence, pub/sub, clustering
- **Memcached** - simple, high-performance, multi-threading
- **Hazelcast** - distributed in-memory data grid

Cloud Provider Solutions:

- AWS ElastiCache/DAX
- Azure Cache for Redis
- GCP Memorystore

HTTP / Web Caching

Leverage HTTP protocol cache-control mechanisms.

Cache Levels:

- Browser caches (client devices)
- Proxy caches (shared caches)
- Reverse proxy caches (front of origin servers)

HTTP Headers:

- `Cache-Control: max-age, no-cache, public/private`
- `ETag` and `Last-Modified` for validation
- Conditional requests

Web Caching Solutions

Open-Source / Self-Hosted:

- **Varnish Cache** - high-performance HTTP accelerator, VCL scripting
- **Nginx** - reverse proxy caching
- **Squid** - caching proxy for bandwidth optimization

Cloud Provider Solutions:

- AWS CloudFront
- Azure Front Door/CDN
- GCP Cloud CDN

Application-Level Caching

Integrate caching logic **directly** into application code.

Advantages:

- Fine-grained control
- Domain-specific strategies
- Understand business logic

Challenges:

- Cache coherency (maintaining consistency)
- Proper cache key design
- Handling cache failures gracefully

Caching Strategies

Cache-Aside (Lazy Loading):

- App checks cache first, loads from DB on miss
- Best for read-heavy with unpredictable patterns

Write-Through:

- Write to cache and DB simultaneously
- Best for consistency with moderate writes

Write-Behind (Write-Back):

- Write to cache first, async to DB
- Best for write-heavy with eventual consistency

Refresh-Ahead:

- Automatically refresh before expiration, best for predictable patterns

Edge Computing

Distribute computation to the **network edge** - closer to users.

Benefits:

- Dramatically reduce latency (eliminate round trips)
- Reduce bandwidth costs (process locally)
- Enable offline operation
- Real-time processing

Physics problem: Data can't travel faster than light - reduce physical distance!

Edge Computing Use Cases

Ideal for:

- Real-time video analytics
- Autonomous vehicles
- Industrial IoT
- Content personalization
- Any scenario where milliseconds matter

Constraints:

- Limited resources vs cloud
- Must handle intermittent connectivity

Edge Solutions

Open-Source / Self-Hosted:

- **KubeEdge** - Kubernetes-based edge platform
- **K3s** - lightweight Kubernetes for edge
- **Apache Edgent** - stream processing for edge devices

Cloud Provider Solutions:

- AWS Lambda@Edge/IoT Greengrass/Wavelength
- Azure IoT Edge/Stack Edge
- GCP Anthos/Edge TPU

Caching: When to Use

- **In-memory caching** for query results, session data, API responses
- **HTTP caching** for static content, rendered pages
- **Cache-aside** for flexible caching with unpredictable patterns
- **Write-through** for strong consistency
- **Edge computing** for IoT, real-time analytics, ultra-low latency
- Set **appropriate TTLs** based on data volatility

Jobs & Scheduling

Jobs & Scheduling

Automate task execution at **specified times** or in **response to events**.

Common Use Cases:

- Data aggregation
- Report generation
- Email delivery
- Database maintenance
- Batch processing

Should not block user-facing operations.

Scheduling Requirements

Must handle:

- Failure scenarios with retry logic
- Job history tracking for auditing
- Prevent duplicate execution
- Visibility into job status and performance

Advanced features:

- Distributed execution across workers
- Dynamic scaling based on queue depth
- Priority-based processing
- Idempotent job design

Time-Based Scheduling (Cron)

Execute jobs at **predetermined intervals** or **specific times**.

Cron Expression Format:

- Minute, hour, day, month, day-of-week patterns
- Example: `0 2 * * *` (every day at 2 AM)

Characteristics:

- Stateless (each execution independent)
- Jobs must be idempotent
- Best for predictable, recurring tasks

Modern Cron Features

Beyond traditional Unix cron:

- Distributed scheduling (only one instance across cluster)
- Missed job handling
- Timezone support
- Integration with monitoring
- Dependency management

Challenges:

- Jobs taking longer than scheduled interval
- Coordinating schedules across distributed systems

Cron Solutions

Open-Source / Self-Hosted:

- **Unix Cron** - standard time-based scheduler
- **Systemd Timers** - modern cron alternative
- **Jenkins** - CI/CD with build scheduling

Cloud Provider Solutions:

- AWS EventBridge
- Azure Functions Timer/Logic Apps
- GCP Cloud Scheduler

Task Queues & Background Jobs

Decouple job submission from execution.

Producer-Consumer Pattern:

- Producers add jobs to queues
- Workers pull jobs for execution
- Prevents long operations from blocking requests
- Enable horizontal scaling (add more workers)

Features:

- Job prioritization (urgent before routine)
- Delayed execution (schedule for future)
- Rate limiting (control throughput)
- Retry with exponential backoff

Task Queue Features

Advanced Capabilities:

- Track job state (pending, running, completed, failed)
- Store results for later retrieval
- Complex workflows (chaining, fan-out/fan-in)
- Scale workers based on queue depth

Benefits:

- Handle variable workloads
- Efficient resource utilization
- Avoid maintaining capacity for peak load

Task Queue Solutions

Open-Source / Self-Hosted:

- **Celery** - distributed Python task queue
- **Sidekiq** - multi-threaded Ruby processor
- **Bull/BullMQ** - Redis-based Node.js queue

Cloud Provider Solutions:

- AWS SQS/Step Functions/Batch
- Azure Queue Storage/Service Bus/Logic Apps/Batch
- GCP Cloud Tasks/Composer/Pub/Sub

Workflow Orchestration

Coordinate **complex, multi-step processes** with dependencies.

Key Capabilities:

- Maintain workflow state
- Handle task failures (retries, compensation)
- Visibility into execution progress
- Conditional logic and branching

Patterns:

- **Orchestration** - central coordinator directs
- **Choreography** - decentralized, event-driven

Workflow Concepts

Directed Acyclic Graphs (DAGs):

- Define dependencies between tasks
- Tasks execute only after prerequisites complete

Advanced Features:

- Dynamic workflows (runtime decisions)
- Long-running processes (days/weeks)
- Distributed transactions (saga patterns)
- Compensating actions for rollbacks

Workflow Solutions

Open-Source / Self-Hosted:

- **Apache Airflow** - Python DAG-based, rich UI (data engineering)
- **Temporal** - durable execution, saga patterns (microservices)
- **Prefect** - modern Python orchestration (data/ML pipelines)

Cloud Provider Solutions:

- AWS Step Functions
- Azure Logic Apps/Data Factory
- GCP Cloud Composer/Workflows

Job Scheduling Libraries

Framework-specific scheduling embedded in applications.

Advantages:

- Lightweight (no external dependencies)
- Simplify deployment
- Minimal operational overhead

Limitations:

- Limited scalability (single instance)
- No distributed coordination
- No job history tracking
- Long-running jobs can block threads

Scheduling: When to Use

- **Cron/Timers** for simple time-based tasks, no dependencies
- **Task Queues** for async processing, decoupling, handling spikes
- **Workflow Orchestration** for complex multi-step processes
- **Managed cloud services** for reduced operational overhead
- **Airflow/Prefect** for data pipelines and ETL
- **Step Functions/Logic Apps** for microservices orchestration

Scheduling Best Practices

Essential patterns:

- Implement proper **retry logic**
- Design for **idempotency** (safe to retry)
- Robust **error handling**
- Comprehensive **monitoring**
- **Dead-letter queues** for failed jobs
- **Timeouts** to prevent stuck jobs